02456 Deep Learning Project

Dense Passage Retrieval with Transformer Encoders



Supervisors:
Michael Alexander Harborg (mialh@dtu.dk)
Anders Vestergaard Nørskov (aveno@dtu.dk)

Abstract

Since its introduction in Vaswani et al. 2017, the Transformer architecture has become widely used in natural language processing, including information retrieval. This project explores how Transformer-based models can be used to retrieve relevant documents through learned vector representations. Students will implement classical retrieval methods such as TF-IDF and BM25, develop key components of a Transformer encoder, and study contrastive learning for Dense Passage Retrieval (DPR). Finally, the different retrieval methods will be evaluated and compared on benchmark datasets.

Key concepts: Transformers, information retrieval, contrastive learning, vector databases.

1 Background

Information retrieval is the task of finding the most relevant documents or passages for a given query. Search engines are a familiar example: a user writes a short query, and the system must decide which documents in a potentially very large collection are most likely to contain the information the user is looking for. A basic challenge is that relevant documents do not always use exactly the same words as the query. For example, a question about "methods for detecting disease from medical images" may be answered by a passage discussing "deep learning for radiological diagnosis", even though relatively few words are shared between the two.

Classical information retrieval methods such as TF-IDF and BM25 approach this problem by assigning scores to documents based largely on the words that appear in the query and in the documents. These methods are simple, efficient, and still widely used. This project will implement these methods from scratch and examine how document frequencies, term frequencies, document lengths, and related statistics influence the ranking of retrieved passages.

The project will then study how deep learning can be used to perform retrieval in a different way. In Dense Passage Retrieval (DPR) (Karpukhin et al. 2020), neural networks are trained to convert both queries and passages into numerical vector representations. During training, the model is shown which passages are relevant for a query and learns to assign these query-passage pairs a higher similarity than pairs containing unrelated passages. After training, all passages can be encoded in advance, and a new query can be answered by finding the passage vectors that are most similar to the vector produced for that query. This makes it possible for the model to learn relationships between queries and passages that are not expressed through exact word overlap.

The core part of the project is the construction of these vector representations. The project will therefore implement important components of a Transformer encoder (Vaswani et al. 2017), including the attention mechanism and the operations surrounding it, and use these components as part of the dense retrieval system. The project will study the contrastive learning setup used in DPR, where relevant and irrelevant passages are compared during training so that relevant query-passage pairs receive more similar representations than unrelated pairs. Once the passage representations have been produced, they will be stored in an index and used for efficient vector-based retrieval.

2 Theory

2.1 TF-IDF and BM25

Suppose we have a set of N documents $\mathcal{D} = \{D_i\}_{i=1}^N$ and receive a text query containing m terms $Q = q_1, \ldots, q_m$. The goal is to rank the relevancy of each document to Q. Let f_{ij} denote the number of times a term q_i occurs in a document D_j. Then the term-frequency of the term q_i with respect to document D_j can be computed as:

$$\mathrm{TF}(q_i, D_j) = \frac{f_{ij}}{\max_k f_{kj}}.$$

To account for the commonality of a term across the set of documents, the inverse document frequency is used which is typically calculated as:

$$\mathrm{IDF}(q_i) = \log\left(\frac{N}{n_{q_i}}\right)$$

where $n_{q_i} = |\{D \in \mathcal{D} : q_i \in D\}|$ is the number of documents that q_i occurs in. Putting these together, the TF-IDF measure becomes:

$$\text{TF-IDF}(q_i, D_j) = \mathrm{TF}(q_i, D_j) \cdot \mathrm{IDF}(q_i)$$

The score of an entire query is commonly implemented as:

$$\mathrm{Score}(Q, D) = \sum_{q \in Q} \text{TF-IDF}(q, D)$$

BM25 improves on TF-IDF in part by saturating the term frequency. This prevents systematically favoring documents that repeatedly use a query word or phrase. Further, BM25 normalizes the document length to prevent longer documents being preferred apriori. To this end, the TF calculation in BM25 is changed to:

$$\mathrm{TF}(q_i, D_j) = \frac{f_{ij}(\kappa + 1)}{f_{ij} + \kappa\left(1 - b + b \cdot \frac{|D_j|}{D_{avg}}\right)}$$

where $D_{avg} = \frac{1}{N}\sum_i |D_i|$ is the average number of terms in a document and $\kappa \in \mathbb{R}_+$, $b \in [0, 1]$ are hyperparameters. Note that as $b \to 0$, length normalization is omitted. Further, as κ increases, the function saturates slower. Lastly, the IDF implementation is also slightly modified:

$$\mathrm{IDF}(q_i) = \log\frac{N + 1}{n_{q_i} + 0.5}$$

This handles the edge case where a term occurring in every document gives 0 under the standard IDF measure. With this, documents that repeat such common terms are slightly favored in the ranking.

2.2 Dense Passage Retrieval

Both TF-IDF and BM25 rely on hand-crafted scoring functions based on word frequencies and document statistics. But what if, instead of explicitly defining how relevance should be measured, we could use deep learning to learn representations of queries and documents in which relevance

can be measured through a simple similarity function? This is the central idea behind Dense Passage Retrieval (DPR) (Karpukhin et al. 2020).

Suppose we have a dataset containing paired documents and queries $\mathcal{D} = \{(D_i, Q_i)\}_{i=1}^{M}$ and let $\mathcal{V}$ denote a vocabulary over tokens present in the dataset.

Let $f_{\boldsymbol{\theta}} : \mathcal{V} \to \mathbb{R}^d$ and $g_{\boldsymbol{\phi}} : \mathcal{V} \to \mathbb{R}^d$ denote transformer encoders (Devlin et al. 2019; Warner et al. 2025) mapping documents and queries, respectively, to a d-dimensional latent space.[1] The goal is to learn the parameters of f and g so that the inner product between paired queries and documents is high and low otherwise. Given a batch of $\mathcal{B}$ document-query pairs, the loss at some pair (D_i, Q_i) takes the form:

$$\ell(\mathcal{B}; \boldsymbol{\theta}, \boldsymbol{\phi}) = -\ln \frac{\exp\left(f_{\boldsymbol{\theta}}(D_i)^\intercal g_{\boldsymbol{\phi}}(Q_i)\right)}{\sum_{j=1}^{|\mathcal{B}|} \exp\left(f_{\boldsymbol{\theta}}(D_j)^\intercal g_{\boldsymbol{\phi}}(Q_i)\right)}.$$

Hence, this loss is minimized when the inner product is large at $i = j$ and small at $i \neq j$. By learning such a latent space, one can retrieve relevant documents that have a low lexical overlap with an input query, addressing a fundamental limitation of TF-IDF and BM25.

3 Data and Model

The primary datasets are FiQA and SQuAD, which contains queries, a collection of text passages, and relevance annotations connecting queries with relevant passages. The dense retriever can be based on a pretrained Transformer encoder such as BERT (Devlin et al. 2019) or ModernBERT (Warner et al. 2025). To get you started on the project, we provide a Github repository with a minimal implementation on this link.

4 Project Goals

- Implement Term Frequency-Inverse Document Frequency (TF-IDF) and the BM25 algorithm from scratch.
- Implement key components of a Transformer encoder.
- Implement key parts of the Dense Passage Retrieval (DPR) training setup.
- Build a vector database and perform dense retrieval.
- Implement retrieval evaluation metrics, including Precision@k, Recall@k, and Mean Reciprocal Rank (MRR), and compare the different retrieval methods using these metrics.
- Qualitatively visualize and examine attention maps in the Transformer encoder to investigate how information is exchanged between tokens.

5 Learning Outcomes

- Be able to implement and explain key components of a Transformer encoder from scratch.
- Understand the attention mechanism and Transformer encoders in general, including layer normalization, residual connections, and positional representations.

[1] If you are unsure about how transformers work, Alammar 2018 and Rush 2018 are excellent resources.

- Understand the contrastive learning setup used in Dense Passage Retrieval. Understand the TF-IDF and BM25 baselines and when lexical retrieval may be preferable to neural retrieval, and vice versa.
- Understand different pooling mechanisms for obtaining fixed-dimensional representations from Transformer outputs.
- Gain familiarity with deep learning frameworks such as PyTorch.

References

[1] Ashish Vaswani et al.
"Attention is all you need".
In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*.
NIPS'17.
Long Beach, California, USA: Curran Associates Inc., 2017,
Pp. 6000–6010.

[2] Vladimir Karpukhin et al.
"Dense Passage Retrieval for Open-Domain Question Answering".
In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
Online: Association for Computational Linguistics, Nov. 2020,
Pp. 6769–6781.
URL: https://aclanthology.org/2020.emnlp-main.550/.

[3] Jacob Devlin et al.
"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding".
In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*.
Ed. by Jill Burstein, Christy Doran, and Thamar Solorio.
Minneapolis, Minnesota: Association for Computational Linguistics, June 2019,
Pp. 4171–4186.
DOI: 10.18653/v1/N19-1423.
URL: https://aclanthology.org/N19-1423/.

[4] Benjamin Warner et al.
"Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference".
In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.
Ed. by Wanxiang Che et al.
Vienna, Austria: Association for Computational Linguistics, July 2025,
Pp. 2526–2547.
ISBN: 979-8-89176-251-0.
DOI: 10.18653/v1/2025.acl-long.127.
URL: https://aclanthology.org/2025.acl-long.127/.

[5] Jay Alammar.
The Illustrated Transformer.
https://jalammar.github.io/illustrated-transformer/.
Blog post. Accessed: 2026-10-13.
2018.

[6] Alexander Rush.
"The Annotated Transformer".
In: *Proceedings of Workshop for NLP Open Source Software (NLP-OSS)*.
Ed. by Eunjeong L. Park et al.
Melbourne, Australia: Association for Computational Linguistics, July 2018,
Pp. 52–60.
URL: https://aclanthology.org/W18-2509/.